

Open Watcom Code Generator Interface

2026

Table of Contents

Introduction	1
General	3
Segments	9
Labels	13
Back Handles	15
Type definitions	17
Procedure Declarations	21
Expressions	23
Leaf Nodes	27
Assignment Operations	29
Arithmetic/logical operations	31
Procedure calls	33
Comparison/short-circuit operations	35
Control flow operations	37
Select and Switch statements.	39
Other	43
Data Generation	47
Front End Routines	51
Debugging Information	63
Registers	71
Miscellaneous	73
A. Pre-defined macros	77
B. Register constants	81
C. Debugging Open Watcom Code Generator	83

Introduction

The code generator (back end) interface is a set of procedure calls. These are divided into following category of routines.

- Code Generation (CG)
- Data Generation (DG)
- Miscellaneous Back End (BE)
- Front end supplied (FE)
- Debugger information (DB)

General

cg_init_info BEInit(cg_switches switches, cg_target_switches targ_switches, uint optsize, proc_revision proc)

Initialize the code generator. This must be the first routine to be called.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>switches</i>	Select code generation options. The options are bits, so may be combined with the bit-wise operator . Options apply to the entire compilation unit. The bit values are defined below.
-----------------	--

<i>targ_switches</i>	Target specific switches. The bit values are defined below.
----------------------	---

<i>optsize</i>	A number between 0 and 100. 0 means optimize for speed, 100 means optimize for size. Anything in between selects a compromise between speed and size.
----------------	---

<i>proc</i>	The target hardware configuration, defined below.
-------------	---

<i>Returns</i>	Information about the code generator revision in a cg_init_info structure, defined below.
----------------	---

<i>Generic Switch</i>	<i>Definition</i>
-----------------------	-------------------

<i>CGSW_GEN_NO_OPTIMIZATION</i>	Turn off optimizations.
--	-------------------------

<i>CGSW_GEN_DBG_NUMBERS</i>	Generate line number debugging information.
------------------------------------	---

<i>CGSW_GEN_FORTRAN_ALIASING</i>	Assume pointers are only used for parameter passing.
---	--

<i>CGSW_GEN_DBG_DF</i>	Generate debugging information in DWARF format.
-------------------------------	---

<i>CGSW_GEN_DBG_CV</i>	Generate debugging information in CodeView format. If neither CGSW_GEN_DBG_DF nor CGSW_GEN_DBG_CV is set, debugging information (if any) is generated in the Watcom format.
-------------------------------	---

<i>CGSW_GEN_RELAX_ALIAS</i>	Assume that a static/extern variable and a pointer to that same variable are not used within the same routine.
------------------------------------	--

<i>CGSW_GEN_DBG_LOCALS</i>	Generate local symbol information for use by a debugger.
-----------------------------------	--

<i>CGSW_GEN_DBG_TYPES</i>	Generate typing information for use by a debugger.
----------------------------------	--

<i>CGSW_GEN_LOOP_UNROLLING</i>	Turn on loop unrolling.
---------------------------------------	-------------------------

<i>CGSW_GEN_LOOP_OPTIMIZATION</i>	Turn on loop optimizations.
--	-----------------------------

CGSW_GEN_INS_SCHEDULING

Turn on instruction scheduling.

CGSW_GEN_MEMORY_LOW_FAILS

Allow the code generator to run out of memory without being able to generate object code (allows the 386 compiler to use EBP as a cache register).

CGSW_GEN_FP_UNSTABLE_OPTIMIZATION

Allow the code generator to perform optimizations that are mathematically correct, but are numerically unstable. E.g. converting division by a constant to a multiplication by the reciprocal.

CGSW_GEN_NULL_DEREF_OK

NULL points to valid memory and may be dereferenced.

CGSW_GEN_FPU_ROUNDING_INLINE

Inline floating-point value rounding (actually truncation) routine when converting floating-point values to integers.

CGSW_GEN_FPU_ROUNDING_OMIT

Omit floating-point value rounding entirely and use FPU default. Results will not be ISO C compliant.

CGSW_GEN_ECHO_API_CALLS

Log each call to the code generator with its arguments and return value. Only available in debug builds.

CGSW_GEN_OBJ_ELF

Emit ELF object files.

CGSW_GEN_OBJ_COFF

Emit COFF object files. For Intel compilers, OMF object files will be emitted in the absence of either switch.

CGSW_GEN_OBJ_ENDIAN_BIG

Emit big-endian object files (COFF or ELF). If CGSW_GEN_OBJ_ENDIAN_BIG is not set, little-endian objects will be generated.

x86 Switch**Definition****CGSW_GEN_I_MATH_INLINE**

Do not check arguments for operators like O_SQRT. This allows the compiler to use some specialty x87 instructions.

CGSW_X86_EZ_OMF

Generate Phar Lap EZ-OMF object files.

CGSW_X86_BIG_DATA

Use segmented pointers (16:16 or 16:32). This defines TY_POINTER to be equivalent to TY_HUGE_POINTER.

CGSW_X86_BIG_CODE

Use inter segment (far) call and return instructions.

CGSW_X86_CHEAP_POINTER

Assume far objects are addressable by one segment value. This must be used in conjunction with CGSW_X86_BIG_DATA. It defines TY_POINTER to be equivalent to TY_FAR_POINTER.

<i>CGSW_X86_FLAT_MODEL</i>	Assume all segment registers address the same base memory.
<i>CGSW_X86_FLOATING_FS</i>	Does FS float (or is it pegged to DGROUP).
<i>CGSW_X86_FLOATING_GS</i>	Does GS float (or is it pegged to DGROUP).
<i>CGSW_X86_FLOATING_ES</i>	Does ES float (or is it pegged to DGROUP).
<i>CGSW_X86_FLOATING_SS</i>	Does SS float (or is it pegged to DGROUP).
<i>CGSW_X86_FLOATING_DS</i>	Does DS float (or is it pegged to DGROUP).
<i>CGSW_X86_USE_32</i>	Generate code into a use32 segment (versus use16).
<i>CGSW_X86_INDEXED_GLOBALS</i>	Generate all global and static variable references as an offset past EBX.
<i>CGSW_X86_WINDOWS</i>	Generate 16-bit Windows prolog/epilog sequences for callbacks and routines.
<i>CGSW_X86_CHEAP_WINDOWS</i>	Generate 16-bit Windows prolog/epilog sequences assuming for callbacks only (cheap).
<i>CGSW_X86_SMART_WINDOWS</i>	Generate 16-bit Windows optimized prolog/epilog sequences assuming DS==SS (smart).
<i>CGSW_GEN_NO_CALL_RET_TRANSFORM</i>	Do not change a CALL followed by a RET into a JMP. This is used for some older overlay managers that cannot handle a JMP to an overlay.
<i>CGSW_X86_CONST_IN_CODE</i>	Generate all constant data into the code segment. This only applies to the internal code generator data, such as floating point constants. The front end decides where its data goes using BESetSeg().
<i>CGSW_X86_NEED_STACK_FRAME</i>	Generate a traceable stack frame. The first instructions will be INC BP if the routine uses a far return instruction, followed by PUSH BP and MOV BP,SP . (ESP and EBP for 386 targets).
<i>CGSW_X86_LOAD_DS_DIRECTLY</i>	Generate code to load DS directly. By default, a call to __GETDS routine is generated.
<i>CGSW_X86_GEN_FWAIT_386</i>	Generate FWAIT instructions on 386 and later CPUs. The 386 never needs FWAIT for data synchronization, but FWAIT may still be needed for accurate exception reporting.

<i>RISC Switch</i>	<i>Definition</i>
<i>CGSW_RISC_ASM_OUTPUT</i>	Print final pseudo-assembly on the console. Debug builds only.
<i>CGSW_RISC_OWL_LOGGING</i>	Log calls to the Object Writer Library
<i>CGSW_RISC_STACK_INIT</i>	Pre-initialize stack variables to a known bit pattern.
<i>CGSW_RISC_EXCEPT_FILTER_USED</i>	Set when SEH (Structured Exception Handling) is used.

The supported `proc_revision` CPU values are:

CPU_86
CPU_186
CPU_286
CPU_386
CPU_486
CPU_586

The supported `proc_revision` FPU values are:

FPU_NONE
FPU_87
FPU_387
FPU_586
FPU_EMU
FPU_E87
FPU_E387
FPU_E586

The supported `proc_revision` WEITEK values are:

WTK_NONE
WTK_1167
WTK_3167
WTK_4167

The following example sets the processor revision information to indicate a 386 with 387 and Weitek 3167.

```
proc_revision proc;  
  
SET_CPU( proc, CPU_386 );  
SET_FPU( proc, FPU_387 );  
SET_WTK( proc, WTK_3167 );
```

The return value structure is defined as follows:


```

typedef struct    cg_init_info {
    unsigned short    revision;    /* contains II_REVISION */
    unsigned short    target;      /* has II_TARG_??? */
} cg_init_info;

enum {
    II_TARG_8086,
    II_TARG_80386,
    II_TARG_STUB,
    II_TARG_CHECK,
    II_TARG_370,
    II_TARG_AXP,
    II_TARG_PPC,
    II_TARG_MIPS
};

```

void BEStart(void)

Start the code generator. Must be called immediately after all calls to BEDefSeg have been made. This restriction is relaxed somewhat for the 80(x)86 code generator. See BEDefSeg for details.

void BEStop(void)

Normal termination of code generator. This must be the second last routine called.

void BEAbort(void)

Abnormal termination of code generator. This must be the second last routine called.

void BEFin(void)

Finalize the code generator. This must be the last routine called.

patch_handle BEPatch(void)

Allocate a patch handle which can be used to create a patchable integer (an integer which will have a constant value provided sometime while the codegen is handling the CGDone call). See CGPatchNode.

void BEPatchInteger(patch_handle hdl, signed_32 value)

Patch the integer corresponding to the given handle to have the given value. This may be called repeatedly with different values, providing CGPatchNode has been called and BEFinPatch has not been called.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>hdl</i>	A patch_handle returned from an earlier invocation of BEPatch which has had a node allocated for it via CGPatchNode. If CGPatchNode has not been called with the handle given, the behaviour is undefined.
-------------------	--

<i>value</i>	A signed 32-bit integer value. This will be the new value of the node which has been associated with the patch handle.
---------------------	--

cg_name BEFiniPatch(patch_handle hdl)

This must be called to free up resources used by the given handle. After this, the handle must not be used again.

Segments

The object file produced by the code generator is composed of various segments. These are defined by the front end. A program may have as many data and code segments as required by the front end. Each segment may be regarded as an individual file of objects, and may be created simultaneously. There is a current segment, selected by `BESetSeg()`, into which all DG routines generate their data. The code for each routine is generated into the segment returned by the `FESegID()` call when it is passed the `cg_sym_handle` for the routine. It is illegal to write data to the code segment for a routine in between the `CGProcDecl` call and the `CGReturn` call.

The following routines are used for initializing, finalizing, defining and selecting segments.

void BEDefSeg(segment_id segid, seg_attr attr, const char *str, uint align)

Define a segment. This must be called after `BEInit` and before `BESetSeg`. For the 80(x)86 code generator, you are allowed to define additional segments after `BESetSeg` if they are:

1. Code Segments
2. PRIVATE data segments.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>segid</i>	A non-negative integer used as an identifier for the segment. It is arbitrarily picked by the front end.
<i>attr</i>	Segment attribute bits, defined below.
<i>str</i>	The name given to the segment.
<i>align</i>	The segment alignment requirements. The code generator will pick the next larger alignment allowed by the object module format. For example, 9 would select paragraph alignment.

<i>Attribute</i>	<i>Definition</i>
-------------------------	--------------------------

<i>EXEC</i>	This is a code segment.
<i>GLOBAL</i>	The segment is accessible to other modules. (versus PRIVATE).
<i>INIT</i>	The segment is statically initialized.
<i>ROM</i>	The segment is read only.
<i>BACK</i>	The code generator may put its data here. One segment must be marked with this attribute. It may not be a COMMON, PRIVATE or EXEC segment. If the front end requires code in the EXEC segment, the <code>CGSW_X86_CONST_IN_CODE</code> switch must be passed to <code>BEInit()</code> .

COMMON All occurrences of this segment will be overlayed. This is used for FORTRAN common blocks.

PRIVATE The segment is non combinable. This is used for far data items.

GIVEN_NAME Normally, the back end feels free to prepend or append strings to the segment name passed in by the front end. This allows a naive front end to specify a constant set of segment names, and have the code generator mangle them in such a manner that they work properly in concert with the set of cg_switches that have been specified (e.g. prepending the module name to the code segments when CGSW_X86_BIG_CODE is specified on the x86). When GIVEN_NAME is specified, the back end outputs the segment name to the object file exactly as given.

THREAD_LOCAL Segment contains thread local data. Such segments may need special handling in executable modules.

segment_id BESetSeg(segment_id segid)

Select the current segment for data generation routines. Code for a routine is always output into the segment returned by FESegID when it is passed the routine symbol handle.

Parameter	Definition
------------------	-------------------

segid	Selects the current segment.
--------------	------------------------------

Returns	The previous current segment.
----------------	-------------------------------

Notes: When emitting data into an EXEC or BACK segment, be aware that the code generator is at liberty to emit code and/or back end data into that segment anytime you make a call to a code generation routine (CG*). Do NOT expect data items to be contiguous in the segment if you have made an intervening CG* call.

segment_id BEGetSeg(void)

Return the current segment for generation routines.

Returns	The current segment.
----------------	----------------------

void BEFlushSeg(segment_id segid)

BEFlushSeg informs the back end that no more code/data will be generated in the specified segment. For code segments, it must be called after the CGReturn() for the final function which is placed in the segment. This causes the code generator to flush all pending information associated with the segment and allows the front end to free all the back handles for symbols which were referenced by the code going into the segment. (The FORTRAN compiler uses this since each function has its own symbol table which is thrown out at the end of the function).

<i>Parameter</i>	<i>Definition</i>
<i>segid</i>	The code segment id.

Labels

The back end uses a **label_handle** for flow of control. Each **label_handle** is a unique code label. These labels may only be used for flow of control. In order to define a label in a data segment, a **back_handle** must be used.

label_handle BNewLabel(void)

Allocate a new control flow label.

Returns A new label_handle.

void BEFiniLabel(label_handle lbl)

Indicate that a label_handle will not be used by the front end anymore. This allows the back end to free some memory at some later stage.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>lbl</i>	A label_handle
-------------------	----------------

Back Handles

A **back_handle** is the front end's handle for a code generator symbol table entry. A **cg_sym_handle** is the code generator's handle for a front end symbol table entry. The back end may call FEBack, passing in any cg_sym_handle that has been passed to it. The front end must allocate a back_handle via BNewBack if one does not exist. Subsequent calls to FEBack should return the same back_handle. This mechanism is used so that the back end does not have to do symbol table searches. For example:

```
back_handle FEBack( SYMPOINTER sym )
{
    if( sym->back == NULL ) {
        sym->back = BNewBack( sym );
    }
    return( sym->back );
}
```

It is the responsibility of the front end to free each back_handle, via BEFreeBack, when it frees the corresponding cg_sym_handle entry.

A back_handle for a symbol having automatic or register storage duration (auto back_handle) may not be freed until CGReturn is called. A back_handle for a symbol having static storage duration, (static back_handle) may not be freed until BStop is called or until after a BEFlushSeg is done for a segment and the back_handle will never be referenced by any other function.

The code generator will not require a back handle for symbols which are not defined in the current compilation unit.

The front end must define the location of all symbols with static storage duration by passing the appropriate back_handle to DGLabel. It must also reserve the correct amount of space for that variable using DGBytes or DGUBytes.

The front end may also allocate an back_handle with static storage duration that has no cg_sym_handle associated with it (anonymous back_handle) by calling BNewBack(NULL). These are useful for literal strings. These must also be freed after calling BStop.

back_handle BNewBack(cg_sym_handle sym)

Allocate a new back_handle.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>sym</i>	The front end symbol handle to be associated with the back_handle. It may be NULL.
-------------------	--

<i>Returns</i>	A new back_handle.
-----------------------	--------------------

void BEFinBack(back_handle bck)

Indicate that **bck** will never be passed to the back end again, except to BEFreeBack. This allows the code generator to free some memory at some later stage.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>bck</i>	A back_handle.
------------	----------------

void BEFreeBack(back_handle bck)

Free the back_handle **bck**. See the preamble in this section for restrictions on freeing a back_handle.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>bck</i>	A back_handle.
------------	----------------

Type definitions

Base types are defined as constants. All other types (structures, arrays, unions, etc) are simply defined by their length. The base types are:

<i>Type</i>	<i>C type</i>
<i>TY_UINT_1</i>	unsigned char
<i>TY_INT_1</i>	signed char
<i>TY_UINT_2</i>	unsigned short
<i>TY_INT_2</i>	signed short
<i>TY_UINT_4</i>	unsigned long
<i>TY_INT_4</i>	signed long
<i>TY_UINT_8</i>	unsigned long long
<i>TY_INT_8</i>	signed long long
<i>TY_LONG_POINTER</i>	far *
<i>TY_HUGE_POINTER</i>	huge *
<i>TY_NEAR_POINTER</i>	near *
<i>TY_LONG_CODE_PTR</i>	(far *)()
<i>TY_NEAR_CODE_PTR</i>	(near *)()
<i>TY_SINGLE</i>	float
<i>TY_DOUBLE</i>	double
<i>TY_LONG_DOUBLE</i>	long double
<i>TY_INTEGER</i>	int
<i>TY_UNSIGNED</i>	unsigned int
<i>TY_POINTER</i>	*
<i>TY_CODE_PTR</i>	(*)()
<i>TY_BOOLEAN</i>	The result of a comparison or flow operator. May also be used as an integer.

<i>TY_DEFAULT</i>	Used to indicate default conversion
<i>TY_NEAR_INTEGER</i>	The result of subtracting 2 near pointers
<i>TY_LONG_INTEGER</i>	The result of subtracting 2 far pointers
<i>TY_HUGE_INTEGER</i>	The result of subtracting 2 huge pointers

There are two special constants.

TY_FIRST_FREE The first user definable type

TY_LAST_FREE The last user definable type.

void BEDefType(cg_type what, uint align, unsigned_32 len)

Define a new type to the code generator.

<i>Parameter</i>	<i>Definition</i>
<i>what</i>	An integral value greater than or equal to TY_FIRST_FREE and less then or equal to TY_LAST_FREE, used as the type identifier.
<i>align</i>	Currently ignored.
<i>len</i>	The length of the new type.

void BEAliasType(cg_type what, cg_type to)

Define a type to be an alias for an existing type.

<i>Parameter</i>	<i>Definition</i>
<i>what</i>	Will become an alias for an existing type.
<i>to</i>	An existing type.

unsigned_32 BTypeLength(cg_type type)

Return the length of a previously defined type, or a base type.

<i>Parameter</i>	<i>Definition</i>
<i>type</i>	A previously defined type.
<i>Returns</i>	The length associated with the type.

uint BTypeAlign(cg_type type)

Return the alignment requirements of a type. This is always 1 for x86 and 370 machines.

Parameter Definition

type A previously defined type.

Returns The alignment requirements of **type** as declared in BEDefType, or for a base type, as defined by the machine architecture.

Procedure Declarations

void CGProcDecl(cg_sym_handle name, cg_type type)

Declare a new procedure. This must be the first routine to be called when generating each procedure.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	The front end symbol table entry for the procedure. A back_handle will be requested.
-------------	--

<i>type</i>	The return type of the procedure. Use TY_INTEGER for void functions.
-------------	--

void CGParmDecl(cg_sym_handle name, cg_type type)

Declare a new parameter to the current function. The calls to this function define the order of the parameters. This function must be called immediately after calling CGProcDecl. Parameters are defined in left to right order, as defined by the procedure prototype.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	The symbol table entry for the parameter.
-------------	---

<i>type</i>	The type of the parameter.
-------------	----------------------------

label_handle CGLastParm(void)

End a parameter declaration section. This function must be called after the last parameter has been declared. Prior to this function, the only calls the front-end is allowed to make are CGParmDecl and CGAutoDecl.

void CGAutoDecl(cg_sym_handle name, cg_type type)

Declare an automatic variable.

This routine may be called at any point in the generation of a function between the calls to CGProcDecl and CGReturn, but must be called before **name** is passed to CGFEName.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	The symbol table entry for the variable.
-------------	--

<i>type</i>	The type of the variable.
-------------	---------------------------

temp_handle CGTemp(cg_type type)

Yields a temporary with procedure scope. This can be used for things such as iteration counts for FORTRAN do loops, or a variable in which to store the return value of a function. This routine should be used **only if necessary**. It should be used when the front end requires a temporary which persists across a flow of control boundary. Other temporary results are handled by the expression trees.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>type</i>	The type of the new temporary.
--------------------	--------------------------------

<i>Returns</i>	A temp_handle which may be passed to CGTempName. This will be freed and invalidated by the back end when CGReturn is called.
-----------------------	--

Expressions

Expression processing involves building an expression tree in the back end, using calls to CG routines. There are routines to generate leaf nodes, binary and unary nodes, and others. These routines return a handle for a node in a back end tree structure, called a **cg_name**. This handle must be exactly once in a subsequent call to a CG routine. A tree may be built in any order, but a cg_name is invalidated by a call to any CG routine with return type void. The exception to this rule is CGTrash.

There is no equivalent of the C address of operator. All leaf nodes generated for symbols, via CGFEName, CGBackName and CGTempName, yield the address of that symbol, and it is the responsibility of the front end to use an indirection operator to get its value. The following operators are available:

<i>0-ary Operator</i>	<i>C equivalent</i>
<i>O_NOP</i>	N/A
<i>Unary Operator</i>	<i>C equivalent</i>
<i>O_UMINUS</i>	-x
<i>O_COMPLEMENT</i>	x
<i>O_POINTS</i>	(*x)
<i>O_CONVERT</i>	x=y
<i>O_ROUND</i>	Do not use!
<i>O_LOG</i>	log(x)
<i>O_COS</i>	cos(x)
<i>O_SIN</i>	sin(x)
<i>O_TAN</i>	tan(x)
<i>O_SQRT</i>	sqrt(x)
<i>O_FABS</i>	fabs(x)
<i>O_ACOS</i>	acos(x)
<i>O_ASIN</i>	asin(x)
<i>O_ATAN</i>	atan(x)
<i>O_COSH</i>	cosh(x)

<i>O_SINH</i>	$\sinh(x)$
<i>O_TANH</i>	$\tanh(x)$
<i>O_EXP</i>	$\exp(x)$
<i>O_LOG10</i>	$\log_{10}(x)$
<i>O_PARENTHESIS</i>	This operator represents the "strong" parentheses of FORTRAN and C. It prevents the back end from performing certain mathematically correct, but floating point incorrect optimizations. E.g. in the expression "(a*2.4)/2.0", the back end is not allowed constant fold the expression into "a*1.2".

<i>Binary Operator</i>	<i>C equivalent</i>
-------------------------------	----------------------------

<i>O_PLUS</i>	+
<i>O_MINUS</i>	-
<i>O_TIMES</i>	*
<i>O_DIV</i>	/
<i>O_MOD</i>	%
<i>O_AND</i>	&
<i>O_OR</i>	
<i>O_XOR</i>	^
<i>O_RSHIFT</i>	>>
<i>O_LSHIFT</i>	<<
<i>O_COMMA</i>	,
<i>O_TEST_TRUE</i>	(x & y) != 0
<i>O_TEST_FALSE</i>	(x & y) == 0
<i>O_EQ</i>	==
<i>O_NE</i>	!=
<i>O_GT</i>	>
<i>O_LE</i>	<=
<i>O_LT</i>	<
<i>O_GE</i>	>=

<i>O_POW</i>	pow(x, y)
<i>O_ATAN2</i>	atan2(x, y)
<i>O_FMOD</i>	fmod(x, y)
<i>O_CONVERT</i>	See below.

The binary **O_CONVERT** operator is only available on the x86 code generator. It is used for based pointer operations (the result type of the **CGBinary** call must be a far pointer type). It effectively performs a **MK_FP** operation with the left hand side providing the offset portion of the address, and the right hand side providing the segment value. If the right hand side expression is the address of a symbol, or the type of the expression is a far pointer, then the segment value for the symbol, or the segment value of the expression is used as the segment value after the **O_CONVERT** operation.

Short circuit operators C equivalent

<i>O_FLOW_AND</i>	&&
<i>O_FLOW_OR</i>	
<i>O_FLOW_NOT</i>	!

Control flow operators C equivalent

<i>O_GOTO</i>	goto label;
<i>O_LABEL</i>	label;;
<i>O_IF_TRUE</i>	if(x) goto label;
<i>O_IF_FALSE</i>	if(!(x)) goto label;
<i>O_INVOKE_LABEL</i>	GOSUB (Basic)
<i>O_LABEL_RETURN</i>	RETURN (Basic)

The type passed into a CG routine is used by the back end as the type for the resulting node. If the node is an operator node (**CGBinary**, **CGUnary**) the back end will convert the operands to the result type before performing the operation. If the type **TY_DEFAULT** is passed, the code generator will use default conversion rules to determine the resulting type of the node. These rules are the same as the ANSI C value preserving rules, with the exception that characters are not promoted to integers before doing arithmetic operations.

For example, if a node of type **TY_UINT_2** and a node of type **TY_INT_4** are to be added, the back end will automatically convert the operands to **TY_INT_4** before performing the addition. The resulting node will have type **TY_INT_4**.

Leaf Nodes

cg_name CGInteger(signed_32 val, cg_type type)

Create an integer constant leaf node.

<i>Parameter</i>	<i>Definition</i>
<i>val</i>	The integral value.
<i>type</i>	An integral type.

cg_name CGInt64(signed_64 val, cg_type type)

Create an 64-bit integer constant leaf node.

<i>Parameter</i>	<i>Definition</i>
<i>val</i>	The 64-bit integer value.
<i>type</i>	An integral type.

cg_name CGFloat(const char *num, cg_type type)

Create a floating-point constant leaf node.

<i>Parameter</i>	<i>Definition</i>
<i>num</i>	A NULL terminated E format string. (-1.23456E-102)
<i>type</i>	A floating point type.

cg_name CGFName(cg_sym_handle sym, cg_type type)

Create a leaf node representing the address of the back_handle associated with **sym**. If sym represents an automatic variable or a parameter, CGAutoDecl or CGParmDecl must be called before this routine is first used.

<i>Parameter</i>	<i>Definition</i>
<i>sym</i>	The front end symbol.
<i>type</i>	The type to be associated with the value of the symbol.

cg_name CGBackName(back_handle bck, cg_type type)

Create a leaf node which represents the address of the back_handle.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>bck</i>	A back handle.
------------	----------------

<i>type</i>	The type to be associated with the value of the symbol.
-------------	--

cg_name CGTempName(temp_handle temp, cg_type type)

Create a leaf node which yields the address of the temp_handle.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>temp</i>	A temp_handle.
-------------	----------------

<i>type</i>	The type to be associated with the value of the symbol.
-------------	--

Assignment Operations

cg_name CGAssign(cg_name dest, cg_name src, cg_type type)

Create an assignment node.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>dest</i>	The destination address.
-------------	--------------------------

<i>src</i>	The source value.
------------	-------------------

<i>type</i>	The type to which the destination address points.
-------------	---

<i>Returns</i>	The value of the right hand side.
----------------	-----------------------------------

cg_name CGLVAssign(cg_name dest, cg_name src, cg_type type)

Like CGAssign, but yields the address of the destination.

cg_name CGPreGets(cg_op op, cg_name dest, cg_name src, cg_type type)

Used for the C expressions $a += b$, $a /= b$.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>op</i>	The arithmetic operator to be used.
-----------	-------------------------------------

<i>dest</i>	The address of the destination.
-------------	---------------------------------

<i>src</i>	The value of the right hand side.
------------	-----------------------------------

<i>type</i>	The type to which the destination address points.
-------------	---

<i>Returns</i>	The value of the left hand side.
----------------	----------------------------------

cg_name CGLVPreGets(cg_op op, cg_name dest, cg_name src, cg_type type)

Like CGPreGets, but yields the address of the destination.

cg_name CGPostGets(cg_op op, cg_name dest, cg_name src, cg_type type)

Used for the C expressions a++, a--. No automatic scaling is done for pointers.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>op</i>	The operator.
------------------	---------------

<i>dest</i>	The address of the destination
--------------------	--------------------------------

<i>src</i>	The value of the increment.
-------------------	-----------------------------

<i>type</i>	The type of the destination.
--------------------	------------------------------

<i>Returns</i>	The value of the left hand side before the operation occurs.
-----------------------	--

Arithmetic/logical operations

cg_name CGBinary(cg_op op, cg_name left, cg_name right, cg_type type)

Binary operations. No automatic scaling is done for pointer operations.

<i>Parameter</i>	<i>Definition</i>
<i>op</i>	The operator.
<i>left</i>	The value of the left hand side.
<i>right</i>	The value of the right hand side.
<i>type</i>	The result type.
<i>Returns</i>	The value of the result.

cg_name CGUnary(cg_op op, cg_name name, cg_type type)

Unary operations.

<i>Parameter</i>	<i>Definition</i>
<i>op</i>	The operator.
<i>name</i>	The value of operand.
<i>type</i>	The result type.
<i>Returns</i>	The value of the result.

cg_name CGIndex(cg_name name, cg_name by, cg_type type, cg_type ptype)

Obsolete. Do not use.

Procedure calls

call_handle CGInitCall(cg_name name, cg_type type, cg_sym_handle aux_info)

Initiate a procedure call.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	The address of the routine to call.
-------------	-------------------------------------

<i>type</i>	The return type of the routine.
-------------	---------------------------------

<i>aux_info</i>	A handle which the back end may passed to FEAuxInfo to determine the attributes of the call.
-----------------	--

<i>Returns</i>	A call_handle to be passed to the following routines.
----------------	--

void CGAddParm(call_handle call, cg_name name, cg_type type)

Add a parameter to a call_handle. The order of parameters is defined by the order in which they are passed to this routine. Parameters should be added in right to left order, as defined by the procedure call.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>call</i>	A call_handle.
-------------	----------------

<i>name</i>	The value of the parameter.
-------------	-----------------------------

<i>type</i>	The type of the parameter. This type will be passed to FEParamType to determine the actual type to be used when passing the parameter. For instance, characters are usually passes as integers in C.
-------------	--

cg_name CGCall(call_handle call)

Turn a call_handle into a cg_name by performing the call. This may be immediately followed by an optional addition operation, to reference a field in a structure return value. An indirection operator must immediately follow, even if the function has no return value.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>call</i>	A call_handle.
-------------	----------------

<i>Returns</i>	The address of the function return value.
----------------	---

Comparison/short-circuit operations

cg_name CGCompare(cg_op op, cg_name left, cg_name right, cg_type type)

Compare two values.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>op</i>	The comparison operator.
------------------	--------------------------

<i>left</i>	The value of the left hand side.
--------------------	----------------------------------

<i>right</i>	The value of the right hand side.
---------------------	-----------------------------------

<i>type</i>	The type to which to convert the operands to before performing comparison.
--------------------	--

<i>Returns</i>	A TY_BOOLEAN cg_name, which may be passed to a control flow CG routine, or used in an expression as an integral value.
-----------------------	--

Control flow operations

cg_name CGFlow(cg_op op, cg_name left, cg_name right)

Perform short-circuit boolean operations.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>op</i>	An operator.
------------------	--------------

<i>left</i>	A TY_BOOLEAN or integral cg_name.
--------------------	-----------------------------------

<i>right</i>	A TY_BOOLEAN or integral cg_name, or NULL if op is O_FLOW_NOT.
---------------------	--

<i>Returns</i>	A TY_BOOLEAN cg_name.
-----------------------	-----------------------

cg_name CGChoose(cg_name sel, cg_name n1, cg_name n2, cg_type type)

Used for the C expression **sel ? n1 : n2**.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>sel</i>	A TY_BOOLEAN or integral cg_name used as the selector.
-------------------	--

<i>n1</i>	The value to return if sel is non-zero.
------------------	--

<i>n2</i>	The value to return if sel is zero.
------------------	--

<i>type</i>	The type to which convert the result.
--------------------	---------------------------------------

<i>Returns</i>	The value of n1 or n2 depending upon the truth of sel .
-----------------------	--

cg_name CGWarp(cg_name before, label_handle label, cg_name after)

To be used for FORTRAN statement functions.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>before</i>	An arbitrary expression tree to be evaluated before label is called. This is used to assign values to statement function arguments, which are usually temporaries allocated with CGTemp.
----------------------	---

<i>label</i>	A label_handle to invoke via O_CALL_LABEL.
---------------------	--

<i>after</i>	An arbitrary expression tree to be evaluated after label is called. This is used to retrieve the statement function return value.
---------------------	--

Returns The value of **after**. This can be passed to CGEval, to guarantee that nested statement functions are fully evaluated before their parameter variables are reassigned, as in f(1,f(2,3,4),5).

void CG3WayControl(cg_name expr, label_handle lt, label_handle eq, label_handle gt)

Used for the FORTRAN arithmetic if statement. Go to label **lt**, **eq** or **gt** depending on whether **expr** is less than, equal to, or greater than zero.

<i>Parameter</i>	<i>Definition</i>
<i>expr</i>	The selector value.
<i>lt</i>	A label_handle.
<i>eq</i>	A label_handle.
<i>gt</i>	A label_handle.

void CGControl(cg_op op, cg_name expr, label_handle lbl)

Generate conditional and unconditional flow of control.

<i>Parameter</i>	<i>Definition</i>
<i>op</i>	a control flow operator.
<i>expr</i>	A TY_BOOLEAN expression if op is O_IF_TRUE or O_IF_FALSE. NULL otherwise.
<i>lbl</i>	The target label.

void CGBigLabel(back_handle lbl)

Generate a label which may be branched to from a nested procedure or used in NT structured exception handling. Don't use this call unless you **really**, **really** need to. It kills a lot of optimizations.

<i>Parameter</i>	<i>Definition</i>
<i>lbl</i>	A back_handle. There must be a front end symbol associated with this back handle.

void CGBigGoto(back_handle value, int level)

Generate a branch to a label in an outer procedure.

<i>Parameter</i>	<i>Definition</i>
<i>lbl</i>	A back_handle. There must be a front end symbol associated with this back handle.
<i>level</i>	The lexical level of the target label.

Select and Switch statements.

The select routines are used as follows. CGSelOther should always be used even if there is no otherwise/default case.

```
end_label = BNewLabel();

sel_label = BNewLabel();
CGControl( O_GOTO, NULL, sel_label );
sel_handle = CGSelInit();

case_label = BNewLabel();
CGControl( O_LABEL, NULL, case_label );
CGSelCase( sel_handle, case_label, case_value );

    ... generate code associated with "case_value" here.

CGControl( O_GOTO, NULL, end_label ); // or else, fall through
other_label = BNewLabel();
CGControl( O_LABEL, NULL, other_label );
CGSelOther( sel_handle, other_label );

    ... generate "otherwise" code here

CGControl( O_GOTO, NULL, end_label ); // or else, fall through

CGControl( O_LABEL, NULL, sel_label );
CGSelect( sel_handle, sel_expr );

CGControl( O_LABEL, NULL, end_label );
```

sel_handle CGSelInit(void)

Create a sel_handle.

Returns A sel_handle to be passed to the following routines.

void CGSelCase(sel_handle s, label_handle lbl, signed_64 val)

Add a single value case to a select or switch.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>s</i>	A sel_handle obtained from CGSelInit().
-----------------	---

<i>lbl</i>	The label to be associated with the case value.
-------------------	---

<i>val</i>	The case value.
-------------------	-----------------

void CGSelRange(sel_handle s, signed_64 lo, signed_64 hi, label_handle lbl)

Add a range of values to a select. All values are eventually converted into unsigned types to generate the switch code, so lo and hi must have the same sign.

<i>Parameter</i>	<i>Definition</i>
<i>s</i>	A sel_handle obtained from CGSelInit().
<i>lo</i>	The lower bound of the case range.
<i>hi</i>	The upper bound of the case range.
<i>lbl</i>	The label to be associated with the case value.

void CGSelOther(sel_handle s, label_handle lbl)

Add the otherwise case to a select.

<i>Parameter</i>	<i>Definition</i>
<i>s</i>	A sel_handle.
<i>lbl</i>	The label to be associated with the otherwise case.

void CGSelect(sel_handle s, cg_name expr)

Add the select expression to a select statement and generate code. This must be the last routine called for a given select statement. It invalidates the sel_handle.

<i>Parameter</i>	<i>Definition</i>
<i>s</i>	A sel_handle.
<i>expr</i>	The value we are selecting.

void CGSelectRestricted(sel_handle s, cg_name expr, cg_switch_type allowed)

Identical to CGSelect, except that only switch generation techniques corresponding to the set of allowed methods will be considered when determining how to produce code.

<i>Parameter</i>	<i>Definition</i>
<i>s</i>	A sel_handle.
<i>expr</i>	The value we are selecting.
<i>allowed</i>	The allowed methods of generating code. Must be a combination (non-empty) of the following bits:

CG_SWITCH_SCAN
CG_SWITCH_BSEARCH

CG_SWITCH_TABLE

Other

void CGReturn(cg_name name, cg_type type)

Return from a function. This is the last routine that may be called in any routine. Multiple return statements must be implemented with assignments to a temporary variable (CGTemp) and a branch to a label generated just before this routine call.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	The value of the return value, or NULL.
-------------	---

<i>type</i>	The type of the return value. Use TY_INTEGER for void functions.
-------------	--

cg_name CGEval(cg_name name)

Evaluate this expression tree now and assign its value to a leaf node. Used to force the order of operations. This should only be used if necessary. Normally, the expression trees adequately define the order of operations. This usually used to force the order of parameter evaluation.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	The tree to be evaluated.
-------------	---------------------------

<i>Returns</i>	A leaf node containing the value of the tree.
----------------	---

void CGDone(cg_name name)

Generate the tree and throw away the resultant value. For example, CGAssign yields a value which may not be needed, but must be passed to this routine to cause the tree to be generated. This routine invalidates all cg_name handles. After this routine has returned, any pending inline function expansions will have been performed.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	The cg_name to be generated/discarded.
-------------	--

void CGTrash(cg_name name)

Like CGDone, but used for partial expression trees. This routine does not cause all existing cg_names to become invalid.

cg_type CGType(cg_name name)

Returns the type of the given cg_name.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	A cg_name.
-------------	------------

<i>Returns</i>	The type of the cg_name.
----------------	--------------------------

cg_name *CGDuplicate(cg_name name)

Create two copies of a cg_name.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	The cg_name to be duplicated.
-------------	-------------------------------

<i>Returns</i>	A pointer to an array of two new cg_names, each representing the same value as the original. These should be copied out of the array immediately since subsequent calls to CGDuplicate will overwrite the array.
----------------	--

cg_name CGBitMask(cg_name name, byte start, byte len, cg_type type)

Yields the address of a bit field. This address may not really be used except with an indirection operator or as the destination of an assignment operation.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>name</i>	The address of the integral variable containing the bit field.
-------------	--

<i>start</i>	The position of the least significant bit of the bit field. 0 indicates the least significant bit of the host data type.
--------------	--

<i>len</i>	The length of the bit field in bits.
------------	--------------------------------------

<i>type</i>	The integral type of the value containing the bit field.
-------------	--

<i>Returns</i>	The address of the bit field. To reference field2 in the following C structure for a little endian target, use start=4, len=5, and type=TY_INT_2. For a big endian target, start=7.
----------------	---

```
typedef struct {  
    short field1 : 4;  
    short field2 : 5;  
    short field3 : 7;  
}
```

cg_name CGVolatile(cg_name name)

Indicate that the given address points to a volatile location. This back end does not remember this information beyond this node in the expression tree. If an address points to a volatile location, the front end must call this routine each time that address is used.

Parameter Definition

name The address of the volatile location.

Returns A new cg_name representing the same value as name.

cg_name CGCallback(cg_callback func, void *ptr)

When a callback node is inserted into the tree, the code generator will call the given function with the pointer as a parameter when it turns the node into an instruction. This can be used to retrieve order information about the placement of nodes in the instruction stream.

Parameter Definition

func This is a pointer to a function which is compatible with the C type "void (*)(void *)". This function will be called with the second parameter to this function as it's only parameter sometime during the execution of the CGDone call.

ptr This will be a parameter to the function given as the first parameter.

cg_name CGPatchNode(patch_handle hdl, cg_type type)

This prepares a leaf node to hold an integer constant which will be provided sometime during the execution of the CGDone call by means of a BEPatchInteger() call. It is an error to insert a patch node into the tree and not call BEPatchInteger().

Parameter Definition

hdl A handle for a patch allocated with BEPatch().

type The actual type of the node. Must be an integer type.

Data Generation

The following routines generate a data item described at the current location in the current segment, and increment the current location by the size of the generated object.

void DGLabel(back_handle bck)

Generate the label for a given back_handle.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>bck</i>	A back_handle.
------------	----------------

void DGBackPtr(back_handle bck, segment_id segid, signed_32 offset, cg_type type)

Generate a pointer to the label defined by the back_handle.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>bck</i>	A back_handle.
------------	----------------

<i>segid</i>	The segment_id of the segment in which the label for bck will be defined if it has not already been passed to DGLabel.
--------------	---

<i>offset</i>	A value to be added to the generated pointer value.
---------------	---

<i>type</i>	The pointer type to be used.
-------------	------------------------------

void DGFEPtr(cg_sym_handle sym, cg_type type, signed_32 offset)

Generate a pointer to the label associated with **sym**.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>sym</i>	A cg_sym_handle.
------------	------------------

<i>type</i>	The pointer type to be used.
-------------	------------------------------

<i>offset</i>	A value to be added to the generated pointer value.
---------------	---

void DGInteger(unsigned_32 value, cg_type type)

Generate an integer.

<i>Parameter</i>	<i>Definition</i>
<i>value</i>	An integral value.
<i>type</i>	The integral type to be used.

void DGInteger64(unsigned_64 value, cg_type type)

Generate an 64-bit integer.

<i>Parameter</i>	<i>Definition</i>
<i>value</i>	An 64-bit integer value.
<i>type</i>	The integral type to be used.

void DGFloat(const char *value, cg_type type)

Generate a floating-point constant.

<i>Parameter</i>	<i>Definition</i>
<i>value</i>	An E format string (ie: 1.2345e-134)
<i>type</i>	The floating point type to be used.

void DGChar(char value)

Generate a character constant. Will be translated if cross compiling.

<i>Parameter</i>	<i>Definition</i>
<i>value</i>	A character value.

void DGString(const char *value, uint len)

Generate a character string. Will be translated if cross compiling.

<i>Parameter</i>	<i>Definition</i>
<i>value</i>	Pointer to the characters to put into the segment. It is not necessarily a null terminated string.
<i>len</i>	The length of the string.

void DGBytes(unsigned_32 len, byte *src)

Generate raw binary data.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>src</i>	Pointer to the data.
------------	----------------------

<i>len</i>	The length of the byte stream.
------------	--------------------------------

void DGIBytes(unsigned_32 len, byte pat)

Generate the byte **pat**, **len** times.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>pat</i>	The pattern byte.
------------	-------------------

<i>len</i>	The number of times to repeat the byte.
------------	---

void DGUBytes(unsigned_32 len)

Generate **len** undefined bytes.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>len</i>	The size by which to increase the segment.
------------	--

void DGAlign(uint align)

Align the segment to an **align** byte boundary. Any slack bytes will have an undefined value.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>align</i>	The desired alignment boundary.
--------------	---------------------------------

unsigned_32 DGSeek(unsigned_32 where)

Seek to a location within a segment.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>where</i>	The location within the segment.
--------------	----------------------------------

<i>Returns</i>	The current location in the segment before the seek takes place.
----------------	--

unsigned long DGTell(void)

Returns The current location within the segment.

unsigned long DGBackTell(back_handle bck)

Returns The location of the label within its segment. The label must have been previously generated via DGLabel.

Front End Routines

void FEGenProc(cg_sym_handle sym)

This routine will be called when the back end is generating a tree and encounters a function call having the **call_class** FECALL_GEN_MAKE_CALL_INLINE. The front end must save its current state and start generating code for **sym**. FEGenProc calls may be nested if the code generator encounters an inline within the code for an inline function. The front end should maintain a state stack. It is up to the front end to prevent infinite recursion.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>sym</i>	The cg_sym_handle of the function to be generated.
------------	--

back_handle FEBack(cg_sym_handle sym)

Return, and possibly allocate using BENewBack, a back handle for sym. See the example under "Back Handles" on page 15

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>sym</i>	
------------	--

<i>Returns</i>	A back_handle.
----------------	----------------

segment_id FESegID(cg_sym_handle sym)

Return the segment_id for symbol **sym**. A negative value may be returned to indicate that the symbol is defined in an unknown PRIVATE segment which has been defined in another module. If two symbols have the same negative value returned, the back end assumes that they are both defined in the same (unknown) segment.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>sym</i>	A cg_sym_handle.
------------	------------------

<i>Returns</i>	A segment_id.
----------------	---------------

const char *FEModuleName(void)

<i>Returns</i>	A null terminated string which is the name of the module being compiled. This is usually the file name with path and extension information stripped.
----------------	--

char FEStackCheck(cg_sym_handle sym)

Returns 1 if stack checking required for this routine

unsigned FELexLevel(cg_sym_handle sym)

Returns The lexical level of routine **sym**. This must be zero for all languages except Pascal. In Pascal, 1 indicates the level of the main program. Each nested procedures adds an additional level.

const char *FENAME(cg_sym_handle sym)

Returns A NULL terminated character string which is the name of **sym**. A null string should be returned if the symbol has no name. NULL should never be returned.

const char *FEExtName(cg_sym_handle sym, int request)

Returns A various kind in dependency on request parameter.

Request parameter Returns

EXTN_BASENAME NULL terminated character string which is the name of **sym**. A null string should be returned if the symbol has no name. NULL should never be returned.

EXTN_PATTERN NULL terminated character string which is the pattern for symbol name decoration. '*' is replaced by symbol name. '^' is replaced by its upper case equivalent. '!' is replaced by its lower case equivalent. '#' is replaced by '@nnn' where nnn is decimal number representing total size of all function parameters. If an '\' is present, the character following is used literally.

EXTN_PRMSIZE Returns int value which represent size of all parameters when symbol is function.

cg_type FEParmType(cg_sym_handle func, cg_sym_handle parm, cg_type type)

Returns The type to which to promote an argument with a given type before passing it to a procedure. Type will be a dealiased type.

int FETrue(void)

Returns The value of TRUE. This is normally 1.

char FEMoreMem(size_t size)

Release memory for the back end to use.

<i>Parameter</i>	<i>Definition</i>
<i>size</i>	is the amount of memory required
<i>Returns</i>	1 if at least size bytes were released. May always return 0 if memory is not a scarce resource in the host environment.

dbg_type FEDbgType(cg_sym_handle sym)

Returns The dbg_type handle for the symbol **sym**.

fe_attr FEAttr(cg_sym_handle sym)

Return symbol attributes for **sym**. These are bits combinable with the bit-wise or operator |.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>sym</i>	A cg_sym_handle.
------------	------------------

<i>Return value</i>	<i>Definition</i>
---------------------	-------------------

<i>FE_PROC</i>	A procedure.
-----------------------	--------------

<i>FE_STATIC</i>	A static or external symbol.
-------------------------	------------------------------

<i>FE_GLOBAL</i>	Is a global (extern) symbol.
-------------------------	------------------------------

<i>FE_IMPORT</i>	Needs to be imported.
-------------------------	-----------------------

<i>FE_CONSTANT</i>	The symbol is read only.
---------------------------	--------------------------

<i>FE_MEMORY</i>	This automatic variable needs a memory location.
-------------------------	--

<i>FE_VISIBLE</i>	Accessible outside this procedure?
--------------------------	------------------------------------

<i>FE_NOALIAS</i>	No pointers point to this symbol.
--------------------------	-----------------------------------

<i>FE_UNIQUE</i>	This symbol should have an address which is different from all other symbols with the FE_UNIQUE attribute.
-------------------------	--

<i>FE_COMMON</i>	There might be multiple definitions of this symbol in a program, and it should be generated in such a way that all versions of the symbol are merged into one copy by the linker.
-------------------------	---

<i>FE_ADDR_TAKEN</i>	The symbol has had it's address taken somewhere in the program (not necessarily visible to the code generator).
-----------------------------	---

<i>FE_VOLATILE</i>	The symbol is "volatile" (in the C language sense).
---------------------------	---

<i>FE_INTERNAL</i>	The symbol is not at file scope.
---------------------------	----------------------------------

void FEMessage(fe_msg femsg, void *extra)

Relays information to the front end.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>femsg</i>	Front-end message.
---------------------	--------------------

<i>extra</i>	Extra information. The type and meaning depends on the value of femsg and is indicated below.
---------------------	--

The femsg parameter values and extra information description.

<i>Value</i>	<i>Description</i>
---------------------	---------------------------

<i>FEMSG_INFO_FILE</i>	Informational message about file. extra (void) is ignored.
-------------------------------	--

<i>FEMSG_CODE_SIZE</i>	Code size. Extra (int) is the size of the generated code.
-------------------------------	---

<i>FEMSG_DATA_SIZE</i>	Data size. Extra (int) is the size of the generated data.
-------------------------------	---

<i>FEMSG_ERROR</i>	A back end error message. Extra (char *) is the error message.
---------------------------	--

<i>FEMSG_FATAL</i>	A fatal code generator error. Extra (char *) is the reason for the fatal error. The front end should issue this message and exit immediately to the system.
---------------------------	---

<i>FEMSG_INFO_PROC</i>	Informational message about current procedure. Extra (char *) is a message.
-------------------------------	---

<i>FEMSG_BAD_PARM_REGISTER</i>	Invalid parameter register returned from FEAuxInfo. Extra (int) is position of the offending parameter.
---------------------------------------	---

<i>FEMSG_BAD_RETURN_REGISTER</i>	Invalid return register returned from FEAuxInfo. Extra (aux_handle) is the offending aux_handle.
---	--

<i>FEMSG_REGALLOC_DIED</i>	The register alloc ran out of memory. Extra (cg_sym_handle) is the procedure which was not fully optimized.
-----------------------------------	---

<i>FEMSG_SCOREBOARD_DIED</i>	The register scoreboard ran out of memory. Extra (cg_sym_handle) is the procedure which was not fully optimized.
-------------------------------------	--

<i>FEMSG_PEEPHOLE_FLUSHED</i>	Peep hole optimizer flushed due to lack of memory. (void)
--------------------------------------	---

<i>FEMSG_BACK_END_ERROR</i>	BAD NEWS! Internal compiler error. Extra (int) is an internal error number.
------------------------------------	---

<i>FEMSG_BAD_SAVE</i>	Invalid register modification information return from FEAuxInfo. Extra (aux_handle) is the offending aux_handle.
------------------------------	--

<i>FEMSG_WANT_MORE_DATA</i>	The back end wants more data space. Extra (int) is amount of additional memory needed to run. (DOS real mode hosts only).
------------------------------------	---

<i>FEMSG_BLIP</i>	Blip. Let the world know we're still alive by printing a dot on the screen. This is called approximately every 4 seconds during code generation. (void)
--------------------------	---

FEMSG_BAD_LINKAGE Cannot resolve linkage conventions. 370 only. (sym)

FEMSG_SCHEDULER_DIED Instruction scheduler ran out of memory. Extra (cg_sym_handle) is the procedure which was not fully optimized.

FEMSG_NO_SEG_REGS (Only occurs in the x86 version). The cg_switches did not allow any segment registers to float, but the user has requested a far pointer indirection. Extra (cg_sym_handle) is the procedure which contained the far pointer usage.

FEMSG_SYMBOL_TOO_LONG Given symbol is too long and is truncated to maximum permitted length for current module output format. Extra (cg_sym_handle) is the symbol which was truncated.

void *FEAuxInfo(void *extra, aux_class class)

relay information to back end

Parameter	Definition
------------------	-------------------

extra	Extra information. Its type and meaning is determined by the value of class.
--------------	--

class	Defined below.
--------------	----------------

Parameters

Return Value

(cg_sym_handle, FEINF_AUX_LOOKUP)

aux_handle - given a cg_sym_handle, return an aux_handle.

(aux_handle, FEINF_CALL_BYTES)

byte_seq * - A pointer to bytes to be generated instead of a call, or NULL if a call is to be generated.

```
typedef struct byte_seq {
    char    length;
    char    data[ 1 ];
} byte_seq;
```

(aux_handle, FEINF_CALL_CLASS)

call_class * - returns call_class of the given aux_handle. See definitions below.

(short, FEINF_FREE_SEGMENT)

short - A free segment value which is free memory for the code generator to use. The first word at segment:0 is the size of the free memory in bytes. (DOS real mode host only)

(NULL, FEINF_OBJECT_FILE_NAME)

char * - The name of the object file to be generated.

(aux_handle, FEINF_PARM_REGS)

hw_reg_set[] - The set of register to be used as parameters.

(*aux_handle*, *FEINF_RETURN_REG*)

hw_reg_set * - The return register. This is only called if the routine is declared to have the FEINF_SPECIAL_RETURN call_class.

(*NULL*, *FEINF_REVISION_NUMBER*)

int - Front end revision number. Must return II_REVISION.

(*aux_handle*, *FEINF_SAVE_REGS*)

hw_reg_set * - Registers which are preserved by the routine.

(*cg_sym_handle*, *FEINF_SHADOW_SYMBOL*)

cg_sym_handle - An alternate handle for a symbol. Required for FORTRAN. Usually implemented by turning on the LSB of a pointer or MSB of an integer.

(*NULL*, *FEINF_SOURCE_NAME*)

char * - The name of the source file to be put into the object file.

(*cg_sym_handle*, *FEINF_TEMP_LOC_NAME*)

Return one of TEMP_LOC_NO, TEMP_LOC_YES, TEMP_LOC_QUIT. After the back end has assigned stack locations to those temporaries which were not placed in registers, it begins to call FEAuxInfo with this request and passes in the cg_sym_handle for each of those temporaries. If the front end responds with TEMP_LOC_QUIT the back end will stop making FEINF_TEMP_LOC_NAME requests. If the front end responds with TEMP_LOC_YES the back end will then perform a FEINF_TEMP_LOC_TELL request (see next). If the front end returns TEMP_LOC_NO the back end moves onto the next cg_sym_handle in its list.

(*int*, *FEINF_TEMP_LOC_TELL*)

Returns nothing. The 'int' value passed in is the relative position on the stack for the temporary identified by the cg_sym_handle passed in from the previous FEINF_TEMP_LOC_NAME. The value for an individual temporary has no meaning, but the difference between two of the values is the number of bytes between the addresses of the temporaries on the stack.

(*void **, *FEINF_NEXT_DEPENDENCY*)

Returns the handle of the next dependency file for which information is available. To start the list off, the back end passes in NULL for the dependency file handle.

(*void **, *FEINF_DEPENDENCY_TIMESTAMP*)

Given the dependency file handle from the last FEINF_NEXT_DEPENDENCY request, return pointer to an unsigned long containing a timestamp value for the dependency file.

(*void **, *FEINF_DEPENDENCY_NAME*)

Given the dependency file handle from the last FEINF_NEXT_DEPENDENCY request, return a pointer to a string containing the name for the dependency file.

(*NULL*, *FEINF_SOURCE_LANGUAGE*)

Returns a pointer to a string which identifies the source language of the pointer. E.g. "C" for C, "FORTRAN" for FORTRAN, "CPP" for C++.

(*cg_sym_handle*, *FEINF_DEFAULT_IMPORT_RESOLVE*)

Only called for imported symbols. Returns a *cg_sym_handle* for another imported symbol which the reference should be resolved to if certain conditions are met (see *FEINF_IMPORT_TYPE* request). If *NULL* or the original *cg_sym_handle* is returned, there is no default import resolution symbol.

(*int*, *FEINF_UNROLL_COUNT*)

Returns a user-specified unroll count, or 0 if the user did not specify an unroll count. The parameter is the nesting level of the loop for which the request is being made. Loops which are not contained inside of other loops are nesting level 1. If this function returns a non-zero value, the loop in question will be unrolled that many times (there will be (count + 1) copies of the body).

x86 Parameters

Return value

(*NULL*, *FEINF_CODE_GROUP*)

char * - The name of the code group.

(*aux_handle*, *FEINF_STRETURN_REG*)

hw_reg_set * - The register which points to a structure return value. Only called if the routine has the *FEINF_SPECIAL_STRUCT_RETURN* attribute.

(*void **, *FEINF_NEXT_IMPORT*)

void * (See notes at end) - A handle for the next symbol to generate a reference to in the object file.

(*void **, *FEINF_IMPORT_NAME*)

char * - The EXTDEF name to generate given a handle

(*void **, *FEINF_NEXT_IMPORT_S*)

void * (See notes at end) - A handle for the next symbol to generate a reference to in the object file.

(*void **, *FEINF_IMPORT_NAME_S*)

Returns a *cg_sym_handle*. The EXTDEF name symbol reference to generate given a handle.

(*void **, *FEINF_NEXT_LIBRARY*)

void * (See notes at end) - Handle for the next library required

(*void **, *FEINF_LIBRARY_NAME*)

char * - The library name to generate given a handle

(*NULL*, *FEINF_DATA_GROUP*)

char * - Used to name DGROUP exactly. *NULL* means use no group at all.

(*segment_id*, *FEINF_CLASS_NAME*)

NULL - Used to name the class of a segment.

(*NULL*, *FEINF_USED_8087*) NULL - Indicate that 8087 instructions were generated.

(*NULL*, *FEINF_STACK_SIZE_8087*)

int - How many 8087 registers are reserved for stack.

(*NULL*, *FEINF_CODE_LABEL_ALIGNMENT*)

char * - An array x, such that x[i] is the label alignment requirements for labels nested within i loops.

(*NULL*, *FEINF_PROEPI_DATA_SIZE*)

int - How much stack is reserved for the prolog hook routine.

(*cg_sym_handle*, *FEINF_IMPORT_TYPE*)

Returns *IMPORT_IS_WEAK*, *IMPORT_IS_LAZY*, *IMPORT_IS_CONDITIONAL* or *IMPORT_IS_CONDITIONAL_PURE*. If the *FEINF_DEFAULT_IMPORT_RESOLVE* request returned a default resolution symbol the back end then performs an *FEINF_IMPORT_TYPE* request to determine the type of the resolution. *IMPORT_IS_WEAK* generates a weak import (the symbol is not searched for in libraries). *IMPORT_IS_LAZY* generates a lazy import (the symbol is searched for in libraries). *IMPORT_IS_CONDITIONAL* is used for eliminating unused virtual functions. The default symbol resolution is used if none of the conditional symbols are referenced/defined by the program. The back end is informed of the list of conditional symbols by the following three aux requests. *IMPORT_IS_CONDITIONAL_PURE* is used for eliminating unused pure virtual functions.

(*cg_sym_handle*, *FEINF_CONDITIONAL_IMPORT*)

Returns void *. Once the back end determines that it has a conditional import, it performs this request to get a conditional list handle which is the head of the list of conditional symbols.

(*void **, *FEINF_CONDITIONAL_SYMBOL*)

Returns a *cg_sym_handle*. Give an conditional list handle, return the front end symbol associated with it.

(*void **, *FEINF_NEXT_CONDITIONAL*)

Given an conditional list handle, return the next conditional list handle. Return NULL at the end of the list.

(*aux_handle*, *FEINF_VIRT_FUNC_REFERENCE*)

Returns void *. When performing an indirect function call, the back end invokes *FEAuxInfo* passing the *aux_handle* supplied with the *CGInitCall*. If the indirect call is referencing a C++ virtual function, the front end should return a magic cookie which is the head of a list of virtual functions that might be invoked by this call. If it is not a virtual function invocation, return NULL.

(void *, FEINF_VIRT_FUNC_NEXT_REFERENCE)

Returns void *. Given the magic cookie returned by the FEINF_VIRT_FUNC_REFERENCE or a previous FEINF_VIRT_FUNC_NEXT_REFERENCE, return the next magic cookie in the list of virtual functions that might be referenced from this indirect call. Return NULL if at the end of the list.

(void *, FEINF_VIRT_FUNC_SYM)

Returns cg_sym_handle. Given a magic cookie from a FEINF_VIRT_FUNC_REFERENCE or FEINF_VIRT_FUNC_NEXT_REFERENCE, return the cg_sym_handle for that entry in the list of virtual functions that might be invoked.

(segment_id, FEINF_PEGGED_REGISTER)

Returns a pointer to a hw_reg_set or NULL. If the pointer is non-NULL and the hw_reg_set is not EMPTY, the hw_reg_set will indicate a segment register that is pegged (pointing) to the given segment_id. The code generator will use this segment register in any references to objects in the segment. If the pointer is NULL or the hw_reg_set is EMPTY, the code generator uses the cg_switches to determine if a segment register is pointing at the segment or if it will have to load one.

Call Class

Meaning

FECALL_GEN_REVERSE_PARDS

Reverse the parameter list.

FECALL_GEN_ABORTS

Routine never returns, optimize caller and callee.

FECALL_GEN_NORETURN

Routine never returns, no optimization, portable.

FECALL_GEN_PARDS_BY_ADDRESS

Pass parameters by reference.

FECALL_GEN_MAKE_CALL_INLINE

Call should be inline. FEGenProc will be called for code sequence when required.

x86 Call Class

Meaning

FECALL_X86_FAR_CALL

Does routine require a far call/return.

FECALL_X86_LOAD_DS_ON_CALL

Load DS from DGROUP prior to call.

FECALL_X86_CALLER_POPS

Caller pops/removes pards from the stack.

FECALL_X86_ROUTINE_RETURN

Routine allocates structure return memory.

FECALL_X86_SPECIAL_RETURN

Routine has non-default return register.

FECALL_X86_NO_MEMORY_CHANGED

Routine modifies no visible statics.

FECALL_X86_NO_MEMORY_READ

Routine reads no visible statics.

FECALL_X86_MODIFY_EXACT

Routine modifies no parameter registers.

FECALL_X86_SPECIAL_STRUCT_RETURN

Routine has special struct return register.

FECALL_X86_NO_STRUCT_REG_RETURNS

Pass 2/4/8 byte structs on stack, as opposed to registers.

FECALL_X86_NO_FLOAT_REG_RETURNS

Return floats as structs.

FECALL_X86_INTERRUPT

Routine is an interrupt routine.

FECALL_X86_NO_8087_RETURNS

No return values in the 8087.

FECALL_X86_LOAD_DS_ON_ENTRY

Load ds with dgroup on entry.

FECALL_GEN_DLL_EXPORT

Is routine an OS/2 export symbol?

FECALL_X86_PROLOG_FAT_WINDOWS

Generate the real mode windows prolog code.

FECALL_X86_GENERATE_STACK_FRAME

Always generate a traceable prolog.

FECALL_X86_EMIT_FUNCTION_NAME

Emit the function name in front of the function in the code segment.

FECALL_X86_GROW_STACK

Emit a call to grow the stack on entry

FECALL_X86_PROLOG_HOOKS

Generate a prolog hook call.

FECALL_X86_EPILOG_HOOKS

Generate an epilog hook call.

FECALL_X86_THUNK_PROLOG

Generate a thunking prolog for routines calling 16 bit code.

FECALL_X86_FAR16_CALL

Performs a 16:16 call in the 386 compiler.

FECALL_X86_TOUCH_STACK

Certain people (who shall remain nameless) have implemented an operating system (which shall remain nameless) that can't be bothered figuring out whether a page reference is in the stack or not. This

attribute forces the first reference to the stack (after a routine prologue has grown it) to be through the SS register.

Debugging Information

These routines generate information about types, symbols, etc.

void DBLineNum(uint no)

Set the current source line number.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>no</i>	Is the current source line number.
-----------	------------------------------------

void DBModSym(cg_sym_handle sym, cg_type indirect)

Define a symbol within the module (file scope).

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>sym</i>	is a front end symbol handle.
------------	-------------------------------

<i>indirect</i>	is the type of indirection needed to obtain the value
-----------------	---

void DBObject(dbg_type tipe, dbg_loc loc)

Define a function as being a member function of a C++ class, and identify the type of the class and the location of the object being manipulated. This function may only be done after the DBModSym for the function.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>tipe</i>	is the debug type of the class that the function is a member of.
-------------	--

<i>loc</i>	is a location expression that evaluates to the address of the object being manipulated by the function (the contents of the 'this' pointer in C++). This parameter is NULL if the routine is a static member function.
------------	--

void DBLocalSym(cg_sym_handle sym, cg_type indirect)

As DBModSym but for local (routine scope) symbols.

void DBGenSym(cg_sym_handle sym, dbg_loc loc, int scoped)

Define a symbol either with module scope ('scoped' == 0) or within the current block ('scoped' != 0). This routine supersedes both DBLocalSym and DBModuleSym. The 'loc' parameter is a location expression (explained later) which allows an arbitrary sequence of operations to locate the storage for the symbol.

<i>Parameter</i>	<i>Definition</i>
-------------------------	--------------------------

<i>sym</i>	is a front end symbol handle.
-------------------	-------------------------------

<i>loc</i>	the location expression which is evaluated by the debugger to locate the lvalue of the symbol.
-------------------	--

<i>scoped</i>	whether the symbol is file scoped or not.
----------------------	---

void DBBegBlock(void)

Open a new scope level.

void DBEndBlock(void)

Close the current scope level.

dbg_type DBScalar(const char *name, cg_type tipe)

Defines the string **name** to have type **tipe**.

dbg_type DBScope(const char *name)

define a symbol which "scopes" subsequent symbols. In C, the keywords **enum**, **union**, **struct** may perform this function as in **struct foo**.

dbg_name DBBegName(const char *name, dbg_type scope)

start a type name whose type is yet undetermined

dbg_type DBForward(dbg_name name)

declare a type to be a forward reference

dbg_type DBEndName(dbg_name name, dbg_type tipe)

complete the definition of a type name.

dbg_type DBArray(dbg_type index, dbg_type base)

define a C array type

dbg_type DBIntArray(unsigned_32 hi, dbg_type base)

define a C array type

dbg_type DBSubRange(signed_32 lo, signed_32 hi, dbg_type base)

define an integer range type

dbg_type DBPtr(cg_type ptr_type, dbg_type base)

declare a pointer type

dbg_type DBBasedPtr(cg_type ptr_type, dbg_type base, dbg_loc seg_loc)

declare a based pointer type. The 'seg_loc' parameter is a location expression which evaluates to the base address for the pointer after the indirection has been performed. Before the location expression is evaluated, the current lvalue of the pointer symbol associated with this type is pushed onto the expression stack (needed for based on self pointers).

dbg_struct DBBegStruct(void)

start a structure type definition

void DBAddField(dbg_struct st, unsigned_32 off, const char *nm, dbg_type base)

add a field to a structure

void DBAddBitField(dbg_struct st, unsigned_32 off, byte strt, byte len, const char *nm, dbg_type base)

add a bit field to a structure

void DBAddLocField(dbg_struct st, dbg_loc loc, uint attr, byte strt, byte len, const char *nm, dbg_type base)

Add a field or bit field to a structure with a generalized location expression 'loc'. The location expression should assume the the address of the base of the structure has already been pushed onto the debugger's evaluation stack. The 'attr' parameter contains a zero or more of the following attributes or'd together:

<i>Attribute</i>	<i>Definition</i>
------------------	-------------------

<i>FIELD_ATTR_INTERNAL</i>	the field is internally generated by the compiler and would not be normally visible to the user.
----------------------------	--

<i>FIELD_ATTR_PUBLIC</i>	the field has the C++ 'public' attribute.
--------------------------	---

<i>FIELD_ATTR_PROTECTED</i>	the field has the C++ 'protected' attribute.
-----------------------------	--

<i>FIELD_ATTR_PRIVATE</i>	the field has the C++ 'private' attribute.
---------------------------	--

If the field being described is `_not_` a bit field, the 'len' parameter should be set to zero.

void DBAddInheritance(dbg_struct st, dbg_type inherit, dbg_loc adjust)

Add the fields of an inherited structure to the current structure being defined.

<i>Parameter</i>	<i>Definition</i>
------------------	-------------------

<i>st</i>	the <code>dbg_struct</code> handle for the structure currently being defined.
-----------	---

<i>inherit</i>	the <code>dbg_type</code> of a previously defined structure which is being inherited.
----------------	---

<i>adjust</i>	a location expression which evaluates to a value which is the amount to adjust the field offsets by in the inherited structure to access them in the current structure. The base address of the symbol associated with the structure type is pushed onto the location expression stack before the expression is evaluated.
---------------	--

dbg_type DBEndStruct(dbg_struct st)

end a structure definition

dbg_enum DBBegEnum(cg_type tipe)

begin defining an enumerated type

void DBAddConst(dbg_enum en, const char *nm, signed_32 val)

add a symbolic constant to an enumerated type

void DBAddConst64(dbg_enum en, const char *nm, signed_64 val)

add a symbolic 64-bit integer constant to an enumerated type

dbg_type DBEndEnum(dbg_enum en)

finish declaring an enumerated type

dbg_proc DBBegProc(cg_type call_type, dbg_type ret)

begin the a current procedure

void DBAddParm(dbg_proc pr, dbg_type tipe)

declare a parameter to the procedure

dbg_type DBEndProc(proc_list *pr)

end the current procedure

dbg_type DBFtnType(const char *name, dbg_ftn_type tipe)

declare a fortran COMPLEX type

dbg_type DBCharBlock(unsigned_32 len)

declare a type to be a block of length **len** characters

dbg_type DBIndCharBlock(back_handle len, cg_type len_type, int off)

declare a type to be a block of characters. The length is found at run-time at back_handle **len** + offset **off**. The integral type of the back_handle location is **len_type**

dbg_type DBLocCharBlock(dbg_loc loc, cg_type len_type)

declare a type to be a block of characters. The length is found at run-time at the address specified by the location expression **loc**. The integral type of the location is **len_type**

dbg_type DBFtnArray(back_handle dims, cg_type lo_bound_tipe, cg_type num_elts_tipe, int off, dbg_type base)

define a FORTRAN array dimension slice. **dims** is a back handle + offset **off** which will point to a structure at run-time. The structure contains the array low bound (type **lo_bound_tipe**) followed by the number of elements (type **num_elts_tipe**). **base** is the element type of the array.

dbg_type DBDereference(cg_type ptr_type, dbg_type base)

declare a type to need an implicit de-reference to retrieve the value (for FORTRAN parameters)

Notes: This routine has been superceded by the use of location expressions.

dbg_loc DBLocInit(void)

create an initial empty location expression

dbg_loc DBLocSym(dbg_loc loc, cg_sym_handle sym)

push the address of 'sym' on to the expression stack

dbg_loc DBLocTemp(dbg_loc loc, temp_handle tmp)

push the address of 'tmp' on to the expression stack

dbg_loc DBLocConst(dbg_loc loc, unsigned_32 val)

push the constant 'val' on to the expression stack

dbg_loc DBLocOp(dbg_loc loc, dbg_loc_op op, unsigned other)

perform the following list of operations on the expression stack

<i>Operation</i>	<i>Definition</i>
-------------------------	--------------------------

<i>DB_OP_POINTS</i>	take the top of the expression stack and use it as the address in an indirection operation. The result type of the operation is given by the 'other' parameter which must be a cg_type which resolves to either an unsigned_16, unsigned_32, a 16-bit far pointer, or a 32-bit far pointer.
----------------------------	---

<i>DB_OP_ZEX</i>	zero extend the top of the stack. The 'other' parameter is a cg_type which is either 1 byte in size or 2 bytes in size. That size determines how much of the original top of stack value to leave untouched.
-------------------------	--

<i>DB_OP_XCHG</i>	exchange the top of stack value with the stack entry indexed by 'other'.
--------------------------	--

<i>DB_OP_MK_FP</i>	take the top two entries on the stack. Make the second entry the segment value and the first entry the offset value of an address.
---------------------------	--

<i>DB_OP_ADD</i>	add the top two stack entries together.
-------------------------	---

<i>DB_OP_DUP</i>	duplicate the top stack entry.
-------------------------	--------------------------------

<i>DB_OP_POP</i>	pop off (throw away) the top stack entry.
-------------------------	---

void DBLocFini(dbg_loc loc)

the given location expression will not be used anymore.

unsigned DBSrcFile(const char *fname)

add the file name into the list of source files for position info, return handle to this name

Notes: Handle 0 is reserved for base source file name and is added by BE automatically during initialization.

void DBSrcCue(unsigned fno, unsigned line, unsigned col)

add source position info for the appropriate source file

Registers

The `hw_reg_set` type is an abstract data type capable of representing any combination of machine registers. It must be manipulated using the following macros. A parameter `c`, `c1`, `c2`, etc. indicate a register constant such as `HW_EAX` must be used. Anything else must be a variable of type `hw_reg_set`.

The following are used for static initialization.

```
HW_D_1( c1 )
HW_NotD_1( c1 )
HW_D_2( c1, c2 )
HW_NotD_2( c1, c2 )
HW_D_3( c1, c2, c3 )
HW_NotD_3( c1, c2, c3 )
HW_D_4( c1, c2, c3, c4 )
HW_NotD_4( c1, c2, c3, c4 )
HW_D_5( c1, c2, c3, c4, c5 )
HW_NotD_5( c1, c2, c3, c4, c5 )
HW_D( c1 )
HW_NotD( c1 )

hw_reg_set regs[] = {
    /* the EAX register */
    HW_D( HW_EAX ),
    /* all registers except EDX and EBX */
    HW_NotD_2( HW_EDX, HW_EBX )
};
```

The following are to build registers dynamically.

<i>Macro</i>	<i>Usage</i>
<i>HW_CEqual(a, c)</i>	Is a equal to c
<i>HW_COvlap(a, c)</i>	Does a overlap with c
<i>HW_CSubset(a, c)</i>	Is c subset of a
<i>HW_CAsgn(dst, c)</i>	Assign c to dst
<i>HW_CTurnOn(dst, c)</i>	Turn on registers c in dst .
<i>HW_CTurnOff(dst, c)</i>	Turn off registers c in dst .
<i>HW_COnlyOn(a, c)</i>	Turn off all registers except c in dst .
<i>HW_Equal(a, b)</i>	Is a equal to b
<i>HW_Ovlap(a, b)</i>	Does a overlap with b

<i>HW_Subset(a, b)</i>	Is b subset of a
<i>HW_Asgn(dst, b)</i>	Assign b to dst
<i>HW_TurnOn(dst, b)</i>	Turn on registers b in dst .
<i>HW_TurnOff(dst, b)</i>	Turn off registers b in dst .
<i>HW_OnlyOn(dst, b)</i>	Turn off all registers except b in dst .

The following example selects the low order 16 bits of any register. that has a low part.

```
hw_reg_set low16( hw_reg_set reg )
{
    hw_reg_set low;

    HW_CAsgn( low, HW_EMPTY );
    HW_CTurnOn( low, HW_AX );
    HW_CTurnOn( low, HW_BX );
    HW_CTurnOn( low, HW_CX );
    HW_CTurnOn( low, HW_DX );
    if( HW_Ovlap( reg, low ) ) {
        HW_OnlyOn( reg, low );
    }
}
```

The following register constants are defined for all targets.

HW_EMPTY The null register set.

HW_UNUSED The set of unused register entries.

HW_FULL All possible registers.

The following example yields the set of all valid machine registers.

```
hw_reg_set reg;

HW_CAsgn( reg, HW_FULL );
HW_CTurnOff( reg, HW_UNUSED );
```

Miscellaneous

I apologize for my lack of consistency in this document. I use the terms function, routine, procedure interchangeably, as well as index, subscript - select, switch - parameter, argument - etc. I come from a multiple language background and will always be hopelessly confused.

The NEXT_IMPORT/NEXT_IMPORT_S/NEXT_LIBRARY are used as follows.

```
handle = NULL;
for( ;; ) {
    handle = FEAuxInfo( handle, NEXT_IMPORT );
    if( handle == NULL )
        break;
    do_something( FEAuxInfo( handle, IMPORT_NAME ) );
}
```

The FREE_SEGMENT request is used as follows.

```
segment = 0;
for( ;; ) {
    segment = FEAuxInfo( segment, FREE_SEGMENT );
    if( segment == NULL )
        break;
    segment_size = *(short *)MK_FP( segment, 0 ) * 16;
    this_is_my_memory_now( MK_FP( segment, 0 ), segment_size );
}
```

The main line in Pascal is defined to be lexical level 1. Add 1 for each nested subroutine level. C style routines are defined to be lexical level 0.

The following types are defined by the code generator header files:

<i>Utility type</i>	<i>Definition</i>
<i>bool</i>	(unsigned char) 0 = false, non-0 = true.
<i>byte</i>	(unsigned char)
<i>int_8</i>	(signed char)
<i>int_16</i>	(signed short)
<i>int_32</i>	(signed long)
<i>signed_8</i>	(signed char)
<i>signed_16</i>	(signed short)
<i>signed_32</i>	(signed long)

<i>uint</i>	(unsigned)
<i>uint_8</i>	(unsigned char)
<i>uint_16</i>	(unsigned short)
<i>uint_32</i>	(unsigned long)
<i>unsigned_8</i>	(unsigned char)
<i>unsigned_16</i>	(unsigned short)
<i>unsigned_32</i>	(unsigned long)
<i>real</i>	(float)
<i>reallong</i>	(double)
<i>pointer</i>	(void *)
Type	Definition
<i>aux_class</i>	(enum) Passed as 2nd parameter to FEAuxInfo.
<i>aux_handle</i>	(void *) A handle used as 1st parameter to FEAuxInfo.
<i>back_handle</i>	(void *) A handle for a back end symbol table entry.
<i>byte_seq</i>	(struct) Passed to back end in response to FEINF_CALL_BYTES FEAuxInfo request.
<i>call_class</i>	(unsigned long) A set of combinable bits indicating the call attributes for a routine.
<i>call_handle</i>	(void *) A handle to be used in CGInitCall, CGAddParm and CGCall.
<i>cg_init_info</i>	(struct) The return value of BEInit.
<i>cg_name</i>	(void *) A handle for a back end expression tree node.
<i>cg_op</i>	(enum) An operator to be used in building expressions.
<i>cg_switches</i>	(unsigned_32) A set of combinable bits indicating the code generator options.
<i>cg_sym_handle</i>	(uint) A handle for a front end symbol table entry.
<i>cg_type</i>	(unsigned short) A code generator type.
<i>fe_attr</i>	(enum) A set of combinable bits indicating symbol attributes.
<i>hw_reg_set</i>	(struct hw_reg_set) A structure representing a hardware register.
<i>label_handle</i>	(void *) A handle for a code generator code label.
<i>linkage_regs</i>	(struct) For 370 linkage conventions.

more_cg_types (enum)

fe_msg (enum) The 1st parameter to FEMessage.

proc_revision (enum) The 3rd parameter to BEInit.

seg_attr (enum) A set of combinable bits indicate the attributes of a segment.

segment_id (int) A segment identifier.

sel_handle (void *) A handle to be used in the CGSel calls.

temp_handle (void *) A handle for a code generator temporary.

Misc Type Definition

HWT hw_reg_part

hw_reg_part (unsigned)

dbg_enum (void *)

dbg_ftn_type (enum)

dbg_name (void *)

dbg_proc (void *)

dbg_struct (void *)

dbg_type (unsigned short)

predefined_cg_types (enum)

A. Pre-defined macros

The following macros are defined by the code generator include files.

```
C_FRONT_END
CPU_MASK
DBG_FWD_TYPE
DBG_NIL_TYPE
DO_FLOATING_FIXUPS
DO_SYM_FIXUPS
FALSE
FIX_SYM_OFFSET
FIX_SYM_RELOFF
FIX_SYM_SEGMENT
FLOATING_FIXUP_BYTE
FORTRAN_FRONT_END
FPU_MASK
FRONT_END_MASK
FUNCS_IN_OWN_SEGMENTS
GET_CPU
GET_FPU
GET_WTK
HWREG_INCLUDED
HW_0
HW_1
HW_2
HW_3
HW_64
HW_Asgn
HW_CAsgn
HW_CEqual
HW_COMMA
HW_COnlyOn
HW_COvlap
HW_CSubset
HW_CTurnOff
HW_CTurnOn
HW_D
HW_D_1
HW_D_2
HW_D_3
HW_D_4
HW_D_5
HW_DEFINE_COMPOUND
HW_DEFINE_GLOBAL_CONST
HW_DEFINE_SIMPLE
HW_Equal
```

HW_ITER
HW_NotD
HW_NotD_1
HW_NotD_2
HW_NotD_3
HW_NotD_4
HW_NotD_5
HW_OnlyOn
HW_Op1
HW_Op2
HW_Op3
HW_Op4
HW_Op5
HW_Ovlap
HW_Subset
HW_TurnOff
HW_TurnOn
II_REVISION
MAX_POSSIBLE_REG
MIN_OP
NULL
NULLCHAR
O_FIRST_COND
O_FIRST_FLOW
O_LAST_COND
O_LAST_FLOW
SEG_EXTRN_FAR
SET_CPU
SET_FPU
SET_WTK
SYM_FIXUP_BYTE
TRUE
TY_HUGE_CODE_PTR
WTK_MASK
_AL
_AX
_BL
_BP
_BX
_CG_H_INCLUDED
_CL
_CMS
_CX
_DI
_DL
_DX
_HOST_INTEGER
_OS
_SI
_TARG_AUX_SHIFT
_TARG_CGSWITCH_SHIFT
far
huge
interrupt

near
offsetof

B. Register constants

The following register constants are defined for x86 targets.

HW_AH
HW_AL
HW_BH
HW_BL
HW_CH
HW_CL
HW_DH
HW_DL
HW_SI
HW_DI
HW_BP
HW_SP
HW_DS
HW_ES
HW_CS
HW_SS
HW_ST0
HW_ST1
HW_ST2
HW_ST3
HW_ST4
HW_ST5
HW_ST6
HW_ST7
HW_FS
HW_GS
HW_AX
HW_BX
HW_CX
HW_DX
HW_EAX
HW_EBX
HW_ECX
HW_EDX
HW_ESI
HW_EDI
HW_ESP
HW_EBP

The following registers are defined for the Alpha AXP target.

HW_R0-HW_R31
HW_D0-HW_D31

HW_W0-HW_W31
HW_B0-HW_B31
HW_F0-HW_F31

The following registers are defined for the PowerPC target.

HW_R0-HW_R31
HW_Q3-HW_Q29
HW_D0-HW_D31
HW_W0-HW_W31
HW_B0-HW_B31
HW_F0-HW_F31

The following registers are defined for the MIPS32 target.

HW_R0-HW_R31
HW_Q2-HW_Q24
HW_D0-HW_D31
HW_W0-HW_W31
HW_B0-HW_B31
HW_F0-HW_F31
HW_FD0-HW_FD30

C. Debugging Open Watcom Code Generator

If you want to use `vc.dbg` command, make sure you have a `tmp` directory in root of used filesystem (see `bld/cg/dumpio.c` for details).

Notes: Make a `s:\tmp` to facilitate debugging in `s:\brad` :) Yeah, it's a cheap and sleazy hack...

If you need to dump something and don't know the routine to call, try `"e/s Dump"` and see what pops up...

Instructions

You can get a dump of instructions for current function via *DumpRange* anytime between *FixEdges* and start of *GenObject*.

You can dump an individual instruction via *DumpIns*

If you need live info for a basic block, find address and call *DumpABlk(block)*.

Symbols

If you need to see a list of symbols, use *DumpSymTab*. To look at one symbol, use *DumpSym*.

Tree Problems

Find the line number of a piece of source near the problem. Do a `"bif { edx == LINENUMBER } DBSrcCue"` to stop near that Go to *CGDone* in order to see what resulting tree is (*DumpTree*) If there is a problem with tree, but not with API calls, do to *DBSrcCue* as above and then break on next appropriate CG API call.

Optimization Problems (Loopopts at all)

Find the ordinal of the problem function in the file (ie 4th function) Do a `"bcnt 4 FixEdges"` in order to stop on 4th call (for example) to *FixEdges* Dump instructions (using *DumpRange*) and see if problem is in trees If not, go to *RegAlloc* and see if problem shows up yet If so, binary search between *FixEdges* and *RegAlloc* to find optimization at fault.

Instruction Select Problems

Go to *RegAlloc* for appropriate function (called once per function when not -od) Find address of instruction which gets translated or handled improperly. (Look in results of *DumpRange* for this address). Do a `"bif { eax == address } ExpandIns"` to look at what we do to this instruction (trace through).

Register Allocation Problem

Instruction Encoding Problem

Go to *RegAlloc* invocation for routine in question. Go to *GenObject* and call *DumpRange*. Find address of instruction that gets encoded incorrectly, and do a "**bif { eax == address } *GenObjCode***" Trace into *GenObjCode* at appropriate time.

A

arithmetic if 38
assignment 29-30

B

back handle 15-16, 47, 51
BEAbort 7
BEAliasType 18
BEDefSeg 9
BEDefType 18
BEFin 7
BEFinBack 16
BEFinLabel 13
BEFlushSeg 10
BEFreeBack 16
BEGetSeg 10
BEInit 3
BENewBack 15
BENewLabel 13
BESetSeg 10
BESStart 7
BESTop 7
BETypeAlign 19
BETypeLength 18
bit fields 44
boolean expressions 35, 37

C

calling conventions 55, 59
CG3WayControl 38
CGAddParm 33
CGAssign 29
CGAutoDecl 21
CGBackName 28
CGBigGoto 38
CGBigLabel 38
CGBinary 31
CGBitMask 44
CGChoose 37
CGCompare 35

CGControl 38
CGDone 43
CGDuplicate 44
CGEval 43
CGFEName 27
CGFloat 27
CGFlow 37
CGIndex 31
CGInitCall 33
CGInt64 27
CGInteger 27
CGLastParm 21
CGLVAssign 29
CGLVPreGets 29
CGParmDecl 21
CGPostGets 30
CGPreGets 29
CGProcDecl 21
CGReturn 43
CGSelCase 39
CGSelect 40
CGSelInit 39
CGSelOther 40
CGSelRange 40
CGTemp 22
CGTempName 28
CGTrash 43
CGType 44
CGUnary 31
CGVolatile 45
CGWarp 37
character 48
control flow 38-40
conversions 25

D

data 47
DBAddBitField 65
DBAddConst 66
DBAddConst64 66
DBAddField 65
DBAddInheritance 66
DBAddLocField 65
DBAddParm 67
DBArray 65
DBBasedPtr 65
DBBegBlock 64
DBBegEnum 66
DBBegName 64

DBBegProc 67
DBBegStruct 65
DBCharBlock 67
DBDereference 68
DBEndBlock 64
DBEndEnum 67
DBEndName 64
DBEndProc 67
DBEndStruct 66
DBForward 64
DBFtnArray 67
DBFtnType 67
DBGenSym 64
DBIndCharBlock 67
DBIntArray 65
DBLineNum 63
DBLocalSym 63
DBLocCharBlock 67
DBLocConst 68
DBLocFini 69
DBLocInit 68
DBLocOp 68
DBLocSym 68
DBLocTemp 68
DBModSym 63
DBObject 63
DBPtr 65
DBScalar 64
DBScope 64
DBSrcCue 69
DBSrcFile 69
DBSubRange 65
DGAlign 49
DGBackPtr 47
DGBackTell 50
DGBytes 49
DGChar 48
DGFEPtr 47
DGFloat 48
DGIBytes 49
DGInteger 48
DGInteger64 48
DGLabel 47
DGSeek 49
DGString 48
DGTell 50
DGUBytes 49

E

error messages 54
expressions 23, 31, 43-45

F

FEAttr 53
FEAuxInfo 55
FEBack 51
FEDbgType 53
FEExtName 52
FEGenProc 51
FELexLevel 52
FEMessage 54
FEModuleName 51
FEMoreMem 52
FENAME 52
FEParmType 52
FESegID 51
FEStackCheck 52
FETrue 52
floating point constant 27, 48
FORTRAN 37-38
functions 21, 33, 43, 51

I

inline procedures 51
integers 27

L

label
 code 13
 data 15, 28, 47

O

operators 23
options 3, 6

V

variables 22, 27-28
volatile 45

P

pascal 52
procedures 21, 33, 43, 51

R

registers 71
relocatable data item 47
routines 21, 33, 43, 51

S

segments 9-10, 47, 49-51
short circuit operations 35, 37
stack probes 52
statement functions 37

T

temporaries 22, 28
types
 predefined 73
typing 17-19, 25, 44